

WK 1

BA LEARNING PLAN

Requirements and Domain Foundations

Business analysis essentials, requirements craft and agile stories

- 01 Business Analysis Foundations**
Role, BACCM, knowledge areas, stakeholders, business case
- 02 Gathering and Documenting Requirements**
Levels, elicitation, quality criteria, BRD, prioritisation
- 03 Agile Requirements Foundations**
Backlog, user stories, INVEST, acceptance criteria
- 04 Worked practice: reorder past meal**
A complete one-page BRD and five user stories with criteria

Week 1 at a glance

Three courses, one deliverable. The courses give you the vocabulary; the deliverable proves you can use it.

Course	The five things to take from it
Business Analysis Foundations	What business analysis is; how the BA differs from PM and PO; the BACCM six concepts; the six knowledge areas; stakeholder analysis
Business Analysis: Gathering and Documenting Requirements	The four requirement levels; functional versus non-functional; elicitation techniques and when to use each; how to write an unambiguous requirement; traceability
Agile Requirements Foundations	Why agile requirements emerge; the backlog hierarchy; user story format and INVEST; acceptance criteria; Definition of Ready and Done

The deliverable

A one-page Business Requirements Document plus five user stories with acceptance criteria, for one feature of an app you use daily. Part 4 of these notes works that deliverable end to end using a food delivery app and the 'reorder past meal' feature, so you have a model to follow and a template to fill in.

EXAM / INTERVIEW TIP

Do not pick an app you love; pick a feature you actually use and can observe yourself using. You need to be able to answer 'what happens when it goes wrong?' from memory, and that only works if you have hit the edge cases yourself.

A realistic week

Day	Focus	Output by end of day
1-2	Business Analysis Foundations	Notes on BACCM, roles, knowledge areas
3-4	Gathering and Documenting Requirements	A filled elicitation-technique table and a draft BRD skeleton
5	Agile Requirements Foundations	Story format, INVEST and acceptance criteria practised
6	Draft the deliverable	First-pass BRD and five stories
7	Review and cut	One page exactly; every story passes INVEST

KEY POINT

The trap in week 1 is producing a long document. A one-page BRD is harder than a ten-page one, and the discipline of cutting is the actual skill being tested. Write it long, then delete.

Contents

Part 1: Business Analysis Foundations	4
1.1 What Business Analysis Is	4
1.2 BA, PM and PO - the Boundaries	4
1.3 BACCM - The Six Core Concepts	5
1.4 The Six Knowledge Areas	5
1.5 Delivery Life Cycles	6
1.6 Stakeholders	6
1.7 Getting to the Real Need	7
1.8 The Business Case	8
Part 2: Gathering and Documenting Requirements	9
2.1 The Four Levels of Requirement	9
2.2 Functional versus Non-functional	9
2.3 What Makes a Requirement Good	10
2.4 Elicitation Techniques	11
2.5 Questioning and Listening	12
2.6 Documenting Requirements	13
2.7 Prioritisation	13
2.8 Verification, Validation and Change	14
Part 3: Agile Requirements Foundations	15
3.1 What Changes in Agile	15
3.2 The Hierarchy	15
3.3 User Stories	16
3.4 Acceptance Criteria	17
3.5 Ready, Done, Refinement and Splitting	17
Part 4: Worked Practice: Reorder Past Meal	19
4.1 How to Structure a One-Page BRD	19

4.2 Worked BRD - One-Tap Reorder	21
4.3 Turning Requirements into User Stories	24
4.4 Five Worked User Stories	25
4.5 Review Checklist Before You Submit	28
4.6 Blank Templates	28

Self-Check Questions

30

PART 1

Business Analysis Foundations

The role, the frameworks and the questions a BA is paid to ask

1

What this part covers

- What business analysis is and the value it adds
- How the BA differs from the project manager and product owner
- The BACCM six core concepts
- The six knowledge areas and where analysis starts and stops
- Delivery life cycles from predictive to adaptive
- Identifying, analysing and engaging stakeholders
- Getting to the real need: five whys, gap analysis, SWOT
- What belongs in a business case

1.1 What Business Analysis Is

Business analysis is the practice of enabling change in an organisation by defining needs and recommending solutions that deliver value to stakeholders.

- **Enabling change** - the job exists because something must be different tomorrow.
- **Defining needs** - the emphasis is the underlying problem, not the solution somebody has already decided on.
- **Recommending solutions** - the BA recommends, the business decides.
- **Delivering value** - if nobody benefits, the analysis failed however tidy the documentation.

KEY POINT

Business analysis is **solution-agnostic**. Valid outcomes include 'change the process, not the software', 'train people differently' and 'do nothing'. Software is one option among several.

1.2 BA, PM and PO – the Boundaries

Role	Question they answer	Accountable for
Business Analyst	What is the problem and what change would solve it?	The right need being understood and clearly specified
Project Manager	How and when will we deliver it, at what cost?	Scope, schedule, budget, risk, resources

Role	Question they answer	Accountable for
Product Owner	What is most valuable to build next?	Maximising product value; owning and ordering the backlog
Architect	How should the solution be structured?	Technical design and non-functional viability
Sponsor	Is this worth funding?	Funding, priority, benefits realisation

EXAM / INTERVIEW TIP

The line that lands in interviews: **the PM manages the project, the BA manages the product definition**. On small teams one person does both, but the accountabilities stay distinct.

1.3 BACCM – The Six Core Concepts

Present in every piece of analysis, at any level, in any methodology. They are interdependent: change one and you must revisit the others.

Concept	Definition	The question it forces
Change	A controlled transformation in response to a need	What exactly will be different afterwards?
Need	A problem, opportunity or constraint with potential value	Why are we doing this at all?
Solution	A specific way of satisfying one or more needs	Does this genuinely address the need?
Stakeholder	Anyone with a relationship to the change, need or solution	Who is affected, who decides, who must be consulted?
Value	The worth of something to a stakeholder	What is the benefit, to whom, and can we measure it?
Context	The circumstances surrounding the change	What environment, culture, regulation and constraints apply?

REMEMBER THIS

C-N-S-S-V-C: Change, Need, Solution, Stakeholder, Value, Context. 'Clever New Solutions Serve Valuable Customers.'

EXAMPLE

Applied to the week 1 practice. **Need:** repeat customers rebuild the same basket every week. **Context:** mobile, one-handed, often at 6pm with a hungry household. **Stakeholders:** customer, restaurant, payments, support. **Solution options:** one-tap reorder, saved favourites, scheduled recurring orders. **Value:** higher repeat rate, shorter checkout. **Change:** new screen, availability check, price-change messaging.

1.4 The Six Knowledge Areas

Knowledge area	Covers	Outputs
Business Analysis Planning and Monitoring	How the analysis itself will be done	BA approach, stakeholder engagement plan
Elicitation and Collaboration	Drawing information out and confirming it	Elicitation results, confirmed notes
Requirements Life Cycle Management	Tracing, prioritising, approving, controlling change	Traceability matrix, baseline, change decisions
Strategy Analysis	Current state, future state, risk, change strategy	Gap analysis, business case, change strategy
Requirements Analysis and Design Definition	Structuring, modelling, verifying, validating; design options	Models, specifications, recommended solution
Solution Evaluation	Measuring delivered value and removing barriers to it	Performance measures, recommendations

EXAM / INTERVIEW TIP

Common question: where does business analysis start and stop? It starts before the project is approved (strategy analysis, business case) and continues after go-live (solution evaluation). It is not a phase.

1.5 Delivery Life Cycles

Life cycle	Requirements	Best when
Predictive (waterfall)	Defined up front, baselined, changed via change control	Stable, well understood, heavily regulated
Iterative	Refined through repeated cycles on the same solution	The solution shape is uncertain
Incremental	Defined per increment, each adding usable capability	Value can be delivered in slices
Adaptive (agile)	Emergent, elaborated just in time, re-prioritised each cycle	High uncertainty, fast feedback, engaged customer

KEY POINT

Predictive fixes scope and lets time and cost move. Adaptive fixes time and cost and lets scope move. Because the backlog is ordered by value, the scope that drops is the least valuable.

1.6 Stakeholders

Finding them

- Follow the process end to end and note everyone who touches it.
- Follow the data: who creates, updates, reads, reports on and archives it.

- Ask every stakeholder who else is affected - snowball outward.
- Do not forget the invisible ones: compliance, legal, security, support desk, training, data privacy, and the end customer.

Power / interest grid

	Low interest	High interest
High influence	Keep satisfied - consult, do not overload	Manage closely - involve, co-create, seek approval
Low influence	Monitor - general communications only	Keep informed - regular updates, useful detail source

RACI

- **R - Responsible:** does the work; can be several people.
- **A - Accountable:** owns and approves the outcome. **Exactly one per activity.**
- **C - Consulted:** two-way input sought before completion.
- **I - Informed:** told afterwards, one-way.

WATCH OUT

Reviewing a RACI, look for three faults: two As on a row (nobody really owns it), no R on a row (the work will not happen), and one person holding A on almost every row (a bottleneck).

1.7 Getting to the Real Need

Five whys

EXAMPLE

'Customers abandon repeat orders.' *Why?* Checkout takes too long. *Why?* They rebuild the basket item by item. *Why?* There is no way to repeat a previous order. *Why?* Order history is read-only. *Why?* History was built for support staff, not customers. The actionable cause is a design assumption, not customer laziness.

Gap analysis

1. Document the current state honestly - process, systems, pain points, metrics.
2. Define the future state in measurable terms.
3. Identify gaps in capability, process, data, technology, skills and policy.
4. Define options to close each gap, then the change strategy.

SWOT

	Helpful	Harmful
Internal	Strengths - what you already have	Weaknesses - gaps and constraints

	Helpful	Harmful
External	Opportunities - shifts you can exploit	Threats - competitors, regulation, risk

Also worth naming

- **Fishbone (Ishikawa)** - groups many contributing causes; better than five whys when the problem has several parallel causes.
- **Context diagram** - the solution as one box with external entities; the fastest way to agree scope boundaries.
- **PESTLE** - external environment scan.

1.8 The Business Case

Section	Content
Problem or opportunity	The need in business terms, with evidence and current cost
Objectives	Measurable outcomes, ideally SMART
Options	Including 'do nothing' as the comparison baseline
Recommendation	Chosen option and the reasoning
Costs and benefits	Build, run, transition, training; benefits quantified and time-phased
Risks and assumptions	What could go wrong, what we take on faith
Success measures	How we will know afterwards whether it worked

- **Feasibility lenses:** technical, operational, economic, schedule and legal.
- **ROI** - net benefit over cost. **Payback** - time to repay. **NPV** - future cash flows in today's money; positive means value added.

PART 2

Gathering and Documenting Requirements

The central craft: eliciting, writing and controlling requirements

2

What this part covers

- The four levels of requirement and the types within them
- Functional versus non-functional, with measurable examples
- Quality criteria and the ambiguity traps to hunt for
- Eleven elicitation techniques and when each is right
- Questioning and active listening
- What goes in a BRD and how it differs from an FRS
- Prioritisation with MoSCoW and friends
- Verification, validation, traceability and change control

2.1 The Four Levels of Requirement

Level	Answers	Example (reorder feature)
Business requirement	Why are we doing this?	Increase the repeat-order rate from 34 to 42 percent within two quarters
Stakeholder requirement	What does this group need to do their job?	A returning customer needs to repeat a past order without rebuilding the basket
Solution - functional	What must the solution do?	The app shall rebuild the basket from a selected past order in a single action
Solution - non-functional	How well must it do it?	The rebuilt basket shall be displayed within 2 seconds on a 4G connection
Transition	What is needed only while we change over?	Backfill 90 days of order history into the new order-history service

KEY POINT

Transition requirements are temporary. Data migration, parallel running, interim manual workarounds and one-off training all stop applying once the change is complete. Examiners love this one.

2.2 Functional versus Non-functional

Functional requirements describe behaviour. Non-functional requirements describe how well the behaviour is performed. NFRs are chronically under-elicited because nobody thinks to ask, and they are expensive to retro-fit.

NFR category	Ask about	Measurable example
Performance	Response time, throughput, concurrency	Basket rebuild completes within 2 seconds for 95 percent of requests
Availability	Uptime, maintenance windows, recovery	99.9 percent uptime during 11am-9pm local trading hours
Security	Authentication, authorisation, encryption, audit	Stored payment tokens encrypted at rest; reorder requires an authenticated session
Usability / accessibility	Learnability, error rates, standards, languages	Meets WCAG 2.1 level AA; reorder completes in no more than 3 taps
Scalability	Growth in users, data, transactions	Sustains 5x normal load during Friday evening peak
Compliance	Regulation, retention, privacy	Age-restricted items still require verification on reorder
Compatibility	Devices, OS versions, integrations	Supports the two most recent major versions of iOS and Android

WATCH OUT

'Fast, secure and user friendly' is a wish, not a requirement. Every NFR needs a **measure, a threshold and a condition**, or it cannot be tested and will be argued about after launch.

Neighbouring concepts not to confuse

Term	Definition
Business rule	A policy that exists whether or not any system does; owned by the business
Assumption	Believed true without proof; must be recorded and validated
Constraint	A restriction the team cannot negotiate - budget, deadline, technology, law
Dependency	One item relies on another; affects sequencing
Risk	Might happen and would affect objectives
Issue	Has already happened and needs resolving

2.3 What Makes a Requirement Good

Characteristic	Test to apply
Atomic	Exactly one need? 'And', 'or' and 'also' are warning signs
Complete	Enough to design and test without asking around?
Consistent	Does it contradict another requirement or rule?

Characteristic	Test to apply
Unambiguous	Could two competent readers read it differently?
Testable	Can you describe the test that proves it is met?
Feasible	Achievable within known constraints?
Necessary	If deleted, would anyone object?
Traceable	Links back to a need and forward to design and tests?
Prioritised	Is its relative importance recorded?

REMEMBER THIS

If you remember only two: **unambiguous** and **testable**. Nearly every other defect surfaces as a failure of one of these.

Ambiguity traps

Trap	Bad	Better
Vague adjective	The basket must load quickly	The basket shall render within 2 seconds on a 4G connection
Undefined pronoun	It notifies them when this happens	The app notifies the customer when an item is unavailable
Passive voice hiding the actor	The order will be validated	The order service validates the order
Compound statement	Validate and save and notify	Split into three separately testable statements
Design disguised as need	Add a Reorder button to the history card	The customer must be able to repeat a previous order
Boundary words	Recent orders can be reordered	Orders placed within the last 90 days can be reordered

- **Shall / must** - mandatory. **Should** - desirable, negotiable. **May** - optional. **Will** - a fact about the environment, not a requirement.

2.4 Elicitation Techniques

Technique	Best when	Limitation
Interview (one to one)	You need depth, or the topic is sensitive	Slow; one perspective; bias risk
Facilitated workshop	Several stakeholders must decide together, fast	Expensive; loud voices dominate without facilitation
Observation / shadowing	The real process differs from the documented one	Slow; observer effect; only shows one day

Technique	Best when	Limitation
Document analysis	Systems, manuals, policies or regulations already exist	Documents go stale and are often aspirational
Survey	Many stakeholders, spread out, questions already known	No probing; poor response rates; bad wording ruins the data
Prototype / wireframe	Requirements are hard to describe in the abstract	Anchors thinking early; looks more finished than it is
Brainstorming	You need breadth before narrowing	Needs discipline and filtering
Focus group	You want attitudes from a representative group	Groupthink
Interface analysis	The solution must integrate with other systems	Technical; needs architect input
Competitive analysis	You need to know what users already expect elsewhere	Copying features is not understanding needs
Data / system analysis	You need evidence on volumes, patterns, exceptions	Needs access and data literacy

EXAM / INTERVIEW TIP

Bring a **deliberately imperfect draft** to every session. People correct far more readily than they invent. A wrong process sketch gets you more in ten minutes than an hour of open questions.

Interview discipline

- Open broad ('walk me through a typical order'), then narrow.
- Ask for artefacts - real screenshots, real receipts, real numbers.
- Chase exceptions explicitly: what happens when it goes wrong?
- Reflect back in your own words before moving on.
- Write up within 24 hours and send for confirmation.

2.5 Questioning and Listening

Type	Purpose	Example
Open	Explore and discover	How do you usually order on a weeknight?
Closed	Confirm or quantify	Do you always order from the same three places?
Probing	Go deeper into an answer	You said 'usually' - when is it not the case?
Clarifying	Remove ambiguity	When you say 'my usual', what exactly is included?
Hypothetical	Explore edge cases	What would you do if an item was sold out?
Leading (avoid)	Plants your answer	You'd want a reorder button, wouldn't you?

KEY POINT

Listen for hedges - **usually, normally, in theory, officially**. Every one of them signals an undocumented exception. Chase all of them.

2.6 Documenting Requirements

Document	Purpose	Audience
Business Requirements Document (BRD)	Why we are doing this and what is in scope	Sponsor, stakeholders, delivery leads
Functional Requirements Spec (FRS/SRS)	What the solution must do, in detail	Developers, testers, architects
Use case specification	Step-by-step behaviour including exceptions	Developers, testers
Requirements Traceability Matrix	Links needs to requirements, designs and tests	BA, QA, audit
Data dictionary	One authoritative definition per data element	Developers, data teams
Glossary	Shared vocabulary	Everyone

The reliable requirement template

[Condition] the [actor or system] shall [action] [object] [qualifier].

- **Weak:** Users should be able to see their old orders.
- **Strong:** When an authenticated customer opens Order History, the app shall list all orders placed in the last 90 days, most recent first, within 2 seconds.

Attributes worth recording per requirement

- Unique ID (never reused), source, priority, status, owner, version, and rationale.
- Rationale is the one people skip and the one that saves you when someone later proposes dropping the requirement.

2.7 Prioritisation

MoSCoW	Meaning	Discipline
Must have	No value, or not legal or safe, without it	You would delay launch for it
Should have	Painful to omit but there is a workaround	Disappointment, not failure
Could have	Desirable if time allows	First to be dropped
Won't have this time	Explicitly excluded from this release	Recorded, not forgotten

WATCH OUT

When everything is a Must, force the trade-off: *if this were the only thing missing on launch day, would you delay the launch?* A 'no' means it is not a Must.

- **Kano** - basic expectations, performance attributes, delighters.
- **Weighted scoring** - score against weighted criteria and rank.
- **100-point method** - each stakeholder distributes 100 points; exposes real preference.
- **Value versus effort** - quick two-by-two triage in a workshop.
- **WSJF** - cost of delay divided by job size; used at scale.

2.8 Verification, Validation and Change

	Verification	Validation
Question	Are we building the thing right?	Are we building the right thing?
Checks	Well written, consistent, testable	Meets the business need and delivers value
Activity	Peer review, quality checklist	Stakeholder review, prototype demo, tracing to objectives

Traceability

- **Forward** - every requirement is designed, built and tested.
- **Backward** - nothing is built that nobody asked for. This is how you catch gold plating.
- Traceability is what makes **impact analysis** possible when a change is proposed.

Change control in five steps

1. Change request raised with justification.
2. BA performs impact analysis across requirements, design, tests, data, process and training.
3. Cost, schedule and risk impact estimated.
4. Decision body approves, rejects or defers.
5. Applied, version incremented, traceability updated, parties notified.

KEY POINT

Scope creep is change that arrives informally, unassessed, decided by whoever asked loudest. **Change control** is the same change made visible, costed and consciously decided. The change is not the problem; the invisibility is.

PART 3

Agile Requirements Foundations

Emergent requirements, user stories and testable acceptance criteria

3

What this part covers

- What changes about requirements in an adaptive approach
- The hierarchy from product goal down to task
- User story format, the three Cs and INVEST
- Writing acceptance criteria that pass or fail cleanly
- Definition of Ready and Definition of Done
- Backlog refinement and story splitting

3.1 What Changes in Agile

Aspect	Traditional	Agile
When analysed	Large batch up front	Continuously, just in time
Detail	Uniform across all scope	Deep at the top of the backlog, shallow below
Container	Signed requirements document	Prioritised, ordered product backlog
Unit	Requirement statement	User story with acceptance criteria
Change	Controlled exception	Expected input each cycle
Acceptance	Document sign-off	Working increment demonstrated

KEY POINT

Agile removes the **big up-front document**, not the **analysis**. Someone still finds the real need, engages awkward stakeholders, models the process, uncovers exception paths and writes testable criteria - just continuously and in smaller batches.

3.2 The Hierarchy

Product goal / vision

Make weeknight reordering effortless

Epic

Reorder a past meal

Feature

One-tap reorder from order history

User story

As Ben I want to reorder my last meal in two taps...

Task

Add availability check to cart-build endpoint

One need, expressed at five levels of granularity

- **Epic** spans multiple sprints. **Feature** spans one or two. **Story** fits comfortably inside one. **Task** is a day or less.
- If a 'story' cannot finish inside a sprint, it is an epic and needs splitting.

3.3 User Stories

EXAMPLE

As a [role or persona] I want [capability] so that [benefit].

As a weeknight regular, I want to repeat my last order in a couple of taps, so that I can eat without rebuilding the same basket again.

- The **role** forces a real user, not 'the system' or 'the user'.
- The **capability** states the need, not the widget.
- The **so that** clause is the most valuable and most often dropped. It carries the value, drives prioritisation, and frequently reveals a cheaper way to meet the need.

The three Cs

C	Meaning	Why it matters
Card	The short written story	Small enough that it cannot pretend to be a spec
Conversation	The discussion between PO, team and stakeholders	Where the real requirement is discovered
Confirmation	The acceptance criteria	Turns opinion into an agreed, testable boundary

INVEST

	Criterion	Test
I	Independent	Can it be built without waiting on another story?
N	Negotiable	Is it a starting point for discussion, not a contract?
V	Valuable	Can you name who benefits and how?

	Criterion	Test
E	Estimable	Does the team understand it well enough to size it?
S	Small	Does it fit comfortably in one sprint?
T	Testable	Can you write criteria that unambiguously pass or fail?

WATCH OUT

If a story fails **Estimable**, the cause is usually a genuine unknown. The answer is a timeboxed **spike**, not a guessed number.

3.4 Acceptance Criteria

Acceptance criteria define the conditions for accepting a story. They set the boundary: what is in and, by omission, what is out. Write them before the sprint, with the tester in the room.

Given / When / Then

EXAMPLE

Given I am signed in and my last order contains an item now out of stock

When I tap Reorder on that order

Then the unavailable item is shown struck through with the label 'Not available today', and the remaining items are added to my basket.

Qualities of good criteria

- **Binary** - passes or fails, no partial credit.
- **Implementation independent** - describe outcome and behaviour, not the widget.
- **Cover the unhappy paths** - empty state, invalid input, timeout, permissions, duplicates, cancellation.
- **Written before development starts** - criteria discovered mid-sprint mean the story was not ready.
- **Sized sensibly** - more than about eight criteria usually means several stories.

KEY POINT

Acceptance criteria are specific to one story and change every time. The **Definition of Done** is a standing checklist applied to every story in the product. Do not put DoD items in your criteria.

3.5 Ready, Done, Refinement and Splitting

	Definition of Ready	Definition of Done
Applies	Before a story enters a sprint	Before a story counts as complete
Typical entries	Value clear; criteria written; dependencies known; sized; designs available; no blocking unknowns	Code reviewed; tests pass; NFR standards met; documentation updated; deployed to test; accepted by the PO

Backlog health – DEEP

- Detailed appropriately, Estimated, Emergent, Prioritised.

Splitting patterns – SPIDR

Pattern	How	Reorder-feature example
Spike	Investigate first, then build	Spike the availability API before integrating
Paths	Happy path first, exceptions later	Reorder when everything is available; substitutions later
Interfaces	One channel at a time	iOS first, then Android, then web
Data	A subset of data first	Orders from the last 30 days first, then 90
Rules	Simple rules first	Standard items first; age-restricted items later

WATCH OUT

Split **vertically**. A thin slice through every layer is demonstrable and valuable. Horizontal slices by technical layer deliver nothing until the last one lands. Warning signs a story needs splitting: it contains 'and', it has more than eight criteria, it covers several roles, or nobody can demo it in two minutes.

PART 4

Worked Practice: Reorder Past Meal

A complete one-page BRD and five user stories you can model yours on

4

What this part covers

- How to structure a one-page BRD and what to cut
- A full worked BRD for a food delivery reorder feature
- How to turn business requirements into user stories
- Five complete user stories with acceptance criteria
- A review checklist before you submit
- Blank templates to copy

WATCH OUT

The numbers, stakeholder names and metrics below are **illustrative**. You are an outside user of the app, not an employee, so you cannot know its real figures. Do the same in your own submission: state plausible targets and label them as assumptions rather than presenting invented data as fact. A BA who flags an assumption looks competent; one who invents statistics looks careless.

4.1 How to Structure a One-Page BRD

A BRD answers **why** and **what**, never **how**. The moment you find yourself describing a screen layout or a database table, you have drifted into the functional spec. Keep it at the level a sponsor could read and approve.

Section	Keep	Cut
Document control	Version, author, date, status, approvers	Change history tables
Background / problem	The pain, with evidence and a cost	Company history, product marketing copy
Objectives	Two to four measurable outcomes	Aspirational adjectives
Scope	In-scope bullets and, crucially, out-of-scope bullets	Anything you are unsure about - park it instead
Stakeholders	Role, interest, and what they decide	Full org charts
Business requirements	Numbered, solution-agnostic, five to eight of them	Field-level detail, UI descriptions
Assumptions and constraints	The ones that would change the answer if wrong	Generic project boilerplate
Risks	Three or four with a mitigation each	A full risk register

Section	Keep	Cut
Success measures	How you will know afterwards	Vanity metrics you cannot access

EXAM / INTERVIEW TIP

Two tests before you submit. **The sponsor test:** could someone who has never seen the app read this and understand why it matters? **The developer test:** would a developer complain there is not enough detail here to build from? If the answer to the second is yes, that is correct - detail belongs in the stories, not the BRD.

4.2 Worked BRD – One-Tap Reorder

Business Requirements Document · FoodDash mobile app · 'Reorder past meal' feature · v0.1 draft

Document control	
Author / owner	[Your name], Business Analyst
Version and date	v0.1 draft, [date]
Status	For review
Approvers	Product Manager (Ordering), Head of Customer Experience, Engineering Lead

1. Background and problem statement

A large share of FoodDash customers are repeat buyers who order the same meal from the same restaurant on a regular cycle. Today, repeating a previous order means starting a new order, finding the restaurant, and re-selecting every item and modifier by hand. Order history exists in the app but is read-only - it was built to help support agents answer queries, not to help customers order again. The result is avoidable friction at the exact moment the customer has already decided to buy.

2. Business objectives

Ref	Objective (assumed baselines, to be confirmed with the product team)
OBJ-1	Increase the proportion of monthly active users placing a repeat order from an assumed 34 percent to 42 percent within two quarters of launch
OBJ-2	Reduce median time from app open to checkout for repeat customers from an assumed 3 minutes 10 seconds to under 45 seconds
OBJ-3	Reduce basket abandonment on repeat orders from an assumed 22 percent to under 12 percent

3. Scope

In scope	Out of scope (this release)
Reordering a complete past order from order history in the mobile app (iOS and Android)	Scheduled or recurring orders
Revalidating item availability, price and restaurant opening hours before payment	Reordering from a different branch of the same restaurant
Amending the rebuilt basket before payment	Group orders and split payment
Orders placed within the last 90 days	The web application (phase 2)
Existing payment, allergen and age-restriction controls continuing to apply	Changes to loyalty points or promotional logic

4. Stakeholders

Stakeholder	Interest	Engagement
Repeat customer	Faster ordering with no unpleasant surprises	Research, usability testing

Stakeholder	Interest	Engagement
Product Manager, Ordering	Repeat rate and conversion	Decision maker on scope
Restaurant partners	Accurate orders, no stale menu items	Consulted on availability rules
Payments team	Token validity, chargeback and refund exposure	Consulted
Customer support	Fewer 'wrong item' and 'price changed' contacts	Consulted; trained before launch
Legal / compliance	Age restriction and allergen obligations upheld	Approval required

5. Business requirements

Ref	Requirement	Priority
BR-01	A returning customer must be able to re-create a previous order without re-selecting each item individually	Must
BR-02	The customer must be informed, before payment, of anything that has changed since the original order - item availability, price, or restaurant opening hours	Must
BR-03	The customer must be able to amend the re-created basket before paying	Must
BR-04	Reorder must be available for orders placed within the last 90 days	Should
BR-05	Reorder must not bypass existing age verification, allergen declaration or payment authorisation controls	Must
BR-06	The business must be able to measure how many orders are placed via reorder and how they convert	Should
BR-07	A customer must be able to reorder from a restaurant that is currently closed by being offered the next available delivery window	Could

6. Assumptions, constraints and dependencies

Type	Statement
Assumption	Order history for at least 90 days is already retained and retrievable
Assumption	Stored payment methods remain valid between orders; expired cards are handled by the existing checkout flow
Assumption	Restaurant menus expose current availability through an existing service
Constraint	Must ship within the existing mobile release train; no new payment provider
Constraint	No change to the checkout flow itself - reorder must feed into it
Dependency	Menu availability service must return item-level availability, not just restaurant open or closed

7. Risks

Risk	Mitigation
Menus change often, so reordered baskets are frequently invalid	Revalidate at reorder time and again at checkout; show changes clearly rather than silently correcting
Customers do not notice a price rise and complain after payment	Require explicit acknowledgement of any price change above a defined threshold
Reorder increases accidental duplicate orders	Show a confirmation summary; detect a near-identical order placed within a short window
Age-restricted items slip through a fast path	Route any basket containing restricted items through existing verification

8. Success measures

- Percentage of orders placed through the reorder path, weekly.
- Median time from app open to payment for reorder journeys versus manual journeys.
- Abandonment rate at the availability-change screen.
- Support contacts tagged 'wrong item' or 'price changed' per thousand orders, before and after launch.
- Repeat-order rate among monthly active users, tracked against OBJ-1.

KEY POINT

Notice what the BRD does **not** say: no mention of buttons, screens, API endpoints or database fields. BR-01 says the customer must be able to re-create an order; it does not say 'add a Reorder button'. That leaves the design team free to find the best solution and keeps the document honest about what is actually required.

4.3 Turning Requirements into User Stories

Business requirements say what the business needs. User stories say what a specific person needs to be able to do, small enough to build and test. The mapping is rarely one to one.

Business requirement	Becomes stories
BR-01 Re-create a previous order	US-01 (see history), US-02 (reorder in place)
BR-02 Inform of changes before payment	US-03 (availability), US-04 (price change)
BR-03 Amend before paying	US-05 (edit basket)
BR-05 Controls still apply	Handled through Definition of Done and existing checkout

Define the persona first

EXAMPLE

Busy Ben, 34, weeknight regular. Orders two to three times a week, almost always the same two restaurants. Orders on his phone one-handed between 5.30 and 7pm, often while doing something else. Cares about speed and predictability far more than discovery. Frustration: 'I know exactly what I want and it still takes me three minutes to order it.'

Story-writing checklist

- Does the role name a real person type, not 'user'?
- Does the 'so that' clause state a benefit, not restate the capability?
- Does it pass INVEST - especially Small and Testable?
- Do the criteria cover at least one unhappy path?
- Could a tester write a test from the criteria without asking you a question?
- Have you avoided naming UI widgets in the story itself?

4.4 Five Worked User Stories

US-01 · View past orders

Priority: Must · Estimate: 3 points

Story. As a weeknight regular, I want to see my recent orders with their items and prices, so that I can find the meal I want to repeat without searching the menu again.

Acceptance criteria

#	Given	When	Then
1	Given I am signed in and have placed at least one order in the last 90 days	When I open Order History	Then I see my orders newest first, each showing restaurant name, order date, item names with quantities, and the total I paid
2	Given I am signed in and have placed no orders in the last 90 days	When I open Order History	Then I see a message explaining there are no recent orders, and a link to browse restaurants
3	Given my order history is still loading	When I open Order History	Then I see a loading state, and if loading fails I see a retry option rather than an empty list

Notes: 90-day window comes from BR-04. Pagination beyond the first 20 orders is a separate story.

US-02 · Reorder a past order

Priority: Must · Estimate: 5 points

Story. As a weeknight regular, I want to repeat a past order in a couple of taps, so that I can order my usual without rebuilding the same basket every time.

Acceptance criteria

#	Given	When	Then
1	Given I am viewing a past order in which every item is still available and the restaurant is open	When I choose to reorder it	Then a basket is created containing exactly the same items, quantities and modifiers, and I am taken to the basket summary
2	Given I already have items in an active basket	When I choose to reorder a past order	Then I am warned that my current basket will be replaced, and the reorder only proceeds if I confirm
3	Given the reorder has been created	When the basket summary is displayed	Then it shows the current total, and it is clear which restaurant the order is from
4	Given I am not signed in	When I attempt to reorder	Then I am prompted to sign in, and the reorder completes automatically once I have

Notes: satisfies BR-01. Delivery address and payment reuse the existing checkout flow, so they are not restated here.

US-03 · Handle unavailable items

Priority: Must · Estimate: 5 points

Story. As a weeknight regular, I want to be told immediately which items from my past order are unavailable, so that I am not surprised at checkout or when the food arrives.

Acceptance criteria

#	Given	When	Then
1	Given one or more items in the past order are no longer available	When I choose to reorder it	Then the unavailable items are listed separately and clearly marked as unavailable, and the available items are added to my basket
2	Given an unavailable item has a suggested replacement from the same restaurant	When I view the unavailable items	Then I can add the suggested replacement to my basket in one action, or dismiss it
3	Given every item in the past order is unavailable	When I choose to reorder it	Then no basket is created, and I am told the order cannot be repeated and offered the restaurant's current menu
4	Given the restaurant is closed at the time I reorder	When I choose to reorder	Then I am shown the next available delivery window before any basket is created

Notes: satisfies BR-02. The closed-restaurant criterion partially delivers BR-07; scheduling into that window is out of scope for this release.

US-04 · Show price changes

Priority: Must · Estimate: 3 points

Story. As a weeknight regular, I want to see clearly when prices have changed since my last order, so that I can decide whether to go ahead before I pay.

Acceptance criteria

#	Given	When	Then
1	Given the price of one or more items has changed since the original order	When my reorder basket is created	Then each changed item shows the previous price and the current price, and the difference in the order total is stated
2	Given the order total has increased by more than 10 percent	When I attempt to proceed to payment	Then I must explicitly acknowledge the increase before I can continue
3	Given no prices have changed	When my reorder basket is created	Then no price-change messaging is shown at all

Notes: satisfies BR-02 and mitigates the 'price rise complaint' risk. The 10 percent threshold is an assumption to confirm with the product team.

US-05 · Amend before paying

Priority: Must · Estimate: 3 points

Story. As a weeknight regular, I want to adjust the reordered basket before I pay, so that I can change quantities or drop something I do not want today.

Acceptance criteria

#	Given	When	Then
1	Given a reorder basket has been created	When I view the basket	Then I can change the quantity of any item, remove any item, and add further items from the restaurant's current menu
2	Given I remove every item from the reorder basket	When the basket becomes empty	Then I am returned to the restaurant menu and the empty basket is not carried into checkout
3	Given I have amended the basket	When the totals are displayed	Then the subtotal, delivery fee and total update immediately to reflect my changes

#	Given	When	Then
4	Given the basket contains an age-restricted item	When I proceed to checkout	Then the existing age verification step is applied before payment

Notes: satisfies BR-03; the final criterion protects BR-05.

KEY POINT

Look at what the criteria have in common: every story covers at least one **unhappy path** - empty history, existing basket, everything unavailable, restaurant closed, basket emptied, restricted item. That is what separates a story a developer can build from a story that generates twelve questions on day one of the sprint.

4.5 Review Checklist Before You Submit

The BRD

- Is it genuinely one page, or one page plus tables you could defend?
- Does every business requirement describe a need rather than a design?
- Is there an explicit out-of-scope list? (This is the single most common omission.)
- Are objectives measurable, with a baseline and a target?
- Are assumptions labelled as assumptions rather than stated as fact?
- Could the sponsor read it in three minutes and know whether to fund it?

The stories

- Does every story have a role, a capability and a benefit?
- Does every story pass INVEST?
- Does every story have at least one unhappy-path criterion?
- Is every criterion binary - it either passes or fails?
- Have you kept Definition of Done items (code review, tests, logging) out of the criteria?
- Do the stories together satisfy the business requirements you listed? Trace them.

EXAM / INTERVIEW TIP

Add a two-column traceability table at the end of your submission mapping each business requirement to the stories that satisfy it. It takes five minutes, it demonstrates you understand traceability, and it immediately exposes any requirement you forgot to cover.

4.6 Blank Templates

BRD skeleton

Section	Your content
Document control	Author, version, date, status, approvers
1. Background and problem	Two to four sentences. What hurts, for whom, and what it costs
2. Objectives	OBJ-1 ... OBJ-3, each measurable
3. Scope	In scope / out of scope
4. Stakeholders	Who, their interest, how engaged
5. Business requirements	BR-01 ... BR-On, solution-agnostic, prioritised
6. Assumptions, constraints, dependencies	Only the ones that would change the answer
7. Risks	Three or four, each with a mitigation

Section	Your content
8. Success measures	How you will know it worked

Story skeleton

Field	Your content
Reference and title	US-On Short verb-noun title
Story	As a [persona], I want [capability], so that [benefit]
Priority and estimate	MoSCoW category; story points
Acceptance criteria	Given [context] / When [action] / Then [outcome] - at least one unhappy path
Notes	Which business requirement it satisfies; assumptions; what is deliberately out

Self-Check Questions

Cover the answer, say yours aloud, then compare.

Foundations

Define business analysis in one sentence.

Enabling change in an organisation by defining needs and recommending solutions that deliver value to stakeholders.

Name the six BACCM concepts.

Change, Need, Solution, Stakeholder, Value, Context.

How does the BA differ from the project manager?

The PM is accountable for delivering the project within scope, schedule and budget; the BA is accountable for the right need being understood and clearly specified.

What is the rule about the letter A in a RACI?

Exactly one Accountable per activity.

Why must 'do nothing' appear in a business case?

It is the baseline every other option is compared against, and it makes the cost of inaction visible.

Requirements

Name the four levels of requirement.

Business, stakeholder, solution (functional and non-functional), and transition.

What makes transition requirements distinctive?

They are temporary - they apply only while moving from current to future state, then stop applying.

Give the two most important quality characteristics.

Unambiguous and testable.

Rewrite 'the basket must load quickly' properly.

State a measure, threshold and condition, for example: the rebuilt basket shall be displayed within 2 seconds on a 4G connection for 95 percent of requests.

Explain forward and backward traceability.

Forward confirms every requirement is designed, built and tested. Backward confirms nothing is built that nobody asked for, which is how you catch gold plating.

Everything is marked Must have. What do you do?

Ask whether you would delay launch if that item alone were missing. A no means it is not a Must.

What belongs in a BRD but not an FRS, and vice versa?

The BRD carries the why and the what at business level - problem, objectives, scope, business requirements. The FRS carries the detailed what the solution must do - functional behaviour, data, rules, interfaces. Neither carries the how of technical design.

Agile

What does INVEST stand for?

Independent, Negotiable, Valuable, Estimable, Small, Testable.

What are the three Cs?

Card, Conversation, Confirmation.

Which part of a user story is most often dropped, and why does it matter?

The 'so that' clause. It carries the value, drives prioritisation, and often reveals a cheaper way to meet the same need.

Acceptance criteria versus Definition of Done?

Criteria are specific to one story and change every time; the Definition of Done is a standing checklist applied to every story.

Why split vertically rather than horizontally?

A vertical slice cuts through all layers and is demonstrable and valuable on its own; horizontal layer-by-layer slices deliver nothing until the last one lands.

Name the five SPIDR splitting patterns.

Spike, Paths, Interfaces, Data, Rules.

A story cannot be estimated. What is the correct response?

A timeboxed spike with a specific question and a defined output, then estimate the story afterwards.

EXAM / INTERVIEW TIP

For your mentor session, be ready to defend two decisions: **why you scoped something out**, and **why you wrote a particular acceptance criterion the way you did**. Those two questions reveal whether you understand the material or have just filled in a template.